

Project Report

ADVANCE ENCRYPTION STANDARD (AES-128-bit)

A Register-Pipelined AES-128 Encryption & Decryption Engine in Verilog

Implemented and Verified on Xilinx Vivado

Hardware Description Language: Verilog-2001

Standard Reference: NIST FIPS PUB 197 (Advanced Encryption Standard)

By- Yashi Sharma

Table of Contents

Table of Contents.....	2
1. Project Title	3
2. Project Overview	3
2.1 What is AES-128?	3
2.2 Why Implement AES in Hardware?.....	3
2.3 Where AES Accelerators Are Used	4
3. Design Architecture.....	5
3.1 Top-Level Block Diagram (Description)	5
3.2 Data Flow: Plaintext to Ciphertext.....	5
3.3 Role of Each Module	6
3.4 Key Expansion (roun1_key.v)	6
3.5 SubBytes	7
3.6 ShiftRows	7
3.7 MixColumns.....	8
3.8 AddRoundKey	8
4. Implementation Details.....	9
4.1 Module Hierarchy	9
4.2 Inputs and Outputs	9
4.3 Registers, Combinational Logic, and Clocking Style	9
4.4 Notable Design Decisions.....	10
5. AES Encryption / Decryption Flow.....	11
5.1 Encryption (ENCRYPTION_new.v).....	11
5.2 Decryption (DECRYPTION.v).....	11
6. Waveform Observations.....	12
7. Optimization and Future Improvements.....	13
8. Reference	14

1. Project Title

AES-128: A Fully-Unrolled, Register-Pipelined AES-128 Encryption & Decryption Engine in Verilog HDL

Subtitle: Hardware Realization of the NIST AES-128 Standard — from Galois Field Arithmetic to Cipher Block Output, verified on Xilinx Vivado.

2. Project Overview

2.1 What is AES-128?

The Advanced Encryption Standard (AES) is a symmetric-key block cipher standardized by NIST under FIPS PUB 197 in 2001. It operates on fixed-size 128-bit blocks of data and supports key lengths of 128, 192, or 256 bits. AES-128 — the variant implemented in this project — uses a 128-bit key and processes data through 10 rounds of transformation, where each round (except the last) consists of four operations: SubBytes, ShiftRows, MixColumns, and AddRoundKey. The cipher treats the 128-bit data block as a 4x4 matrix of bytes, called the State, and the round operations manipulate this State matrix to achieve confusion (obscuring the relationship between key and ciphertext) and diffusion (spreading the influence of each plaintext bit across the ciphertext).

AES replaced the older Data Encryption Standard (DES) because DES's 56-bit key became vulnerable to brute-force attacks. AES-128 offers 2^{128} possible keys, making exhaustive search computationally infeasible with current and foreseeable technology, while remaining efficient enough for hardware and embedded implementation.

2.2 Why Implement AES in Hardware?

While AES can be executed in software on any general-purpose processor, a dedicated hardware implementation offers several engineering advantages that matter in real security-critical systems:

- **Throughput and latency:** A hardware datapath can process an entire 128-bit block per fixed number of clock cycles without instruction-fetch or branching overhead, making it dramatically faster than a software loop for high-bandwidth encryption.
- **Power efficiency:** Dedicated combinational logic (S-Boxes, Galois Field multipliers) consumes far less energy per bit encrypted than a general-purpose CPU executing the same algorithm in software — critical for battery-powered and IoT devices.
- **Side-channel resistance:** Software AES implementations are vulnerable to cache-timing and branch-prediction attacks. Hardware datapaths with fixed, data-independent timing (as in this fully-unrolled design) inherently close some of these timing side channels.

- **Isolation and trust:** A hardware security block can operate in a separate, isolated silicon domain (a Root of Trust or Hardware Security Module), keeping cryptographic keys away from a potentially compromised software stack or operating system.
- **Determinism:** Hardware pipelines provide predictable, cycle-accurate latency — essential for real-time encrypted communication links where jitter is unacceptable.

2.3 Where AES Accelerators Are Used

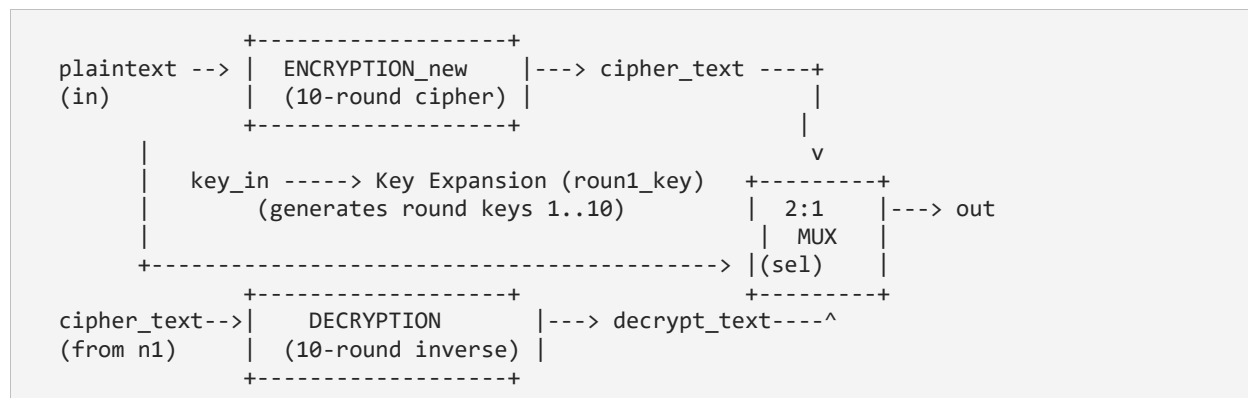
Hardware AES engines, similar in spirit to the one designed in this project, are pervasive across the electronics industry:

Domain	Example Application
Security chips / Crypto ICs	Trusted Platform Modules (TPMs), smart cards, secure elements (SE) for payment and identity
IoT and embedded systems	Encrypting sensor data and firmware updates on resource-constrained microcontrollers
Networking & communication	IPsec, TLS record-layer offload, 5G/Wi-Fi (WPA2/WPA3) link-layer encryption
Storage systems	Self-encrypting drives (SEDs), full-disk encryption controllers
Cloud & data-center hardware	Inline encryption engines in network interface cards (NICs) and HSMs
Automotive & industrial	Secure CAN/Ethernet communication, firmware authentication in ECUs

3. Design Architecture

3.1 Top-Level Block Diagram (Description)

The design is organized as a hierarchical datapath rooted at the AES top module. Conceptually, the system has three major blocks feeding a single output multiplexer:



Both ENCRYPTION_new and DECRYPTION instantiate their own copy of the key-expansion module (roun1_key) internally and run on every clock edge. The DECRYPTION module's data input is wired directly to the ENCRYPTION_new module's output — i.e., the design always decrypts the ciphertext it just produced from the same key, rather than accepting an externally supplied ciphertext. The 1-bit sel signal then chooses, via a registered multiplexer, whether out reflects the freshly computed cipher_text or the recovered decrypt_text. This is a deliberate (if implicit) self-verifying loopback architecture: because decrypt_text should always equal the original plaintext in, the design can validate its own correctness in simulation without an external golden reference.

3.2 Data Flow: Plaintext to Ciphertext

Inside ENCRYPTION_new, the 128-bit State matrix flows through the classic AES pipeline:

1. Initial AddRoundKey — the raw plaintext is XORed with the original 128-bit key (key_in) before any round begins.
2. Rounds 1-9 — each round performs SubBytes -> ShiftRows -> MixColumns -> AddRoundKey, using a fresh round key generated by the key-expansion unit.
3. Round 10 (final round) — performs SubBytes -> ShiftRows -> AddRoundKey only; MixColumns is deliberately omitted in the final round, exactly as specified by FIPS-197, so that the cipher remains invertible.
4. The 128-bit result is registered into cipher_text.

Every one of these eleven (initial + 10 rounds, condensing SubBytes/ShiftRows/MixColumns/AddRoundKey) AES sub-operations is implemented as its own Verilog module with a synchronous output register. This means the design is not a single combinational cloud — it is a deeply staged datapath where data advances by exactly one sub-

operation per clock edge, all the way from n1 (initial AddRoundKey) through n41 (final AddRoundKey) inside ENCRYPTION_new.

3.3 Role of Each Module

Module	Role in the Datapath
AES.v	Top-level wrapper. Instantiates encryption and decryption cores, multiplexes the output with sel, and applies global clk/clr.
ENCRYPTION_new.v	Forward cipher: implements the initial round + 9 main rounds + final round, wiring in round keys from roun1_key.
DECRYPTION.v	Inverse cipher: mirrors the round structure in reverse using inverse S-Box, inverse ShiftRows, and inverse MixColumns.
roun1_key.v	Key Expansion (Key Schedule): expands the 128-bit cipher key into 10 round keys using RotWord, SubWord, and Rcon.
original_key.v / array_of_byte.v	Capture and byte-slice the input 128-bit key into sixteen 8-bit registers for use by the key schedule.
sbox.v / inv_sbox.v	Combinational 256-entry lookup tables implementing the AES forward and inverse S-Box byte substitution.
test_sub_byte.v / inv_sub_byte.v	Apply the S-Box (or inverse S-Box) to all 16 bytes of the State in parallel.
test_shift_row.v / inv_shift_row.v	Cyclically shift each row of the State matrix (forward or inverse direction).
test_column_mixing.v / inv_column_mix.v	Perform the MixColumns / InvMixColumns matrix multiplication in $GF(2^8)$.
multiply_2.v / multiply_3.v	Galois Field multiply-by-2 (xtime) and multiply-by-3 primitives used by MixColumns.
mul_09 / mul_0b / mul_0d / mul_0e.v	Galois Field multiply-by-9/11/13/14 primitives, built from multiply_2, used by InvMixColumns.
add_round_key.v	Bitwise XOR of the State with the current round key (used identically in both encryption and decryption).

3.4 Key Expansion (roun1_key.v)

Purpose: AES does not reuse the original 128-bit key in every round. Instead, a key schedule expands the single cipher key into eleven 128-bit round keys (the original key for the initial AddRoundKey, plus one per round). This is essential cryptographically: reusing the same key for every round would make the cipher far easier to analyze and break.

Round key generation: the key is treated as four 32-bit words ($w_0..w_3$). Each subsequent word is generated as $w_i = w_{(i-4)} \text{ XOR } f(w_{(i-1)})$, where f is the identity function for three out of every four words, and a special transformation for every fourth word. In this design, the key schedule generates 16 new key bytes per round through a chain of XOR assign statements, exactly mirroring this word-recurrence relation (visible in the $r\{N\}_k\{B\}_w\{B\}$ naming throughout roun1_key.v).

RCON usage: the special transformation applied once per round is: rotate the last word's bytes left by one position (RotWord), substitute each byte through the S-Box (SubWord), then XOR only the most-significant byte with a round constant, RCON[round]. The design hardcodes the ten RCON values exactly as specified by the AES standard:

RCON: 01, 02, 04, 08, 10, 20, 40, 80, 1B, 36 (round 1 through round 10)

Word operations: the 'left shift' (RotWord) is implemented by simply re-wiring the four key bytes with a cyclic offset (e.g., $l1_k0 = key13$; $l1_k1 = key14$; ...), the SubWord step instantiates four parallel sbox modules, and the RCON XOR is applied only to the first resulting byte ($rcon1_k0 = s1_k0 \wedge rcon_1$) before all 16 bytes of the new round key are formed through the recursive XOR chain.

Integration with AES rounds: all ten round keys are computed combinatorially in parallel (since each round key only depends on the previous one through a chain of XORs and a few S-Box lookups) and then latched into registers $r1_k0..r10_k15$ on a single clock edge. This means the full key schedule is available one clock cycle after key_in is presented — a useful property, but also the longest single combinational path in the design (ten chained rounds of SubWord + XOR), discussed further in Section 8.

3.5 SubBytes

S-Box transformation: SubBytes is the only non-linear operation in AES and is what gives the cipher its resistance to linear and differential cryptanalysis. Each byte of the State is independently replaced by a corresponding byte from a fixed 256-entry substitution table (the S-Box), which was derived mathematically by taking the multiplicative inverse of the byte in $GF(2^8)$ and then applying an affine transformation.

Byte substitution concept: substitution provides confusion — it makes the relationship between the secret key and the ciphertext as complex and non-linear as possible, so that no simple algebraic shortcut can recover the key from known plaintext/ciphertext pairs.

Implementation approach: rather than computing the $GF(2^8)$ inverse and affine map in logic, this design implements the S-Box as a pure combinational lookup table (sbox.v) using a 256-way case statement, which Vivado synthesizes efficiently into LUTs / block ROM. The wrapper module `test_sub_byte.v` slices the 128-bit State into sixteen bytes, routes each through its own dedicated sbox instance in parallel (16 instances total per SubBytes stage), and registers the recombined 128-bit result. `inv_sub_byte.v` / `inv_sbox.v` mirror this exactly for decryption using the inverse table.

3.6 ShiftRows

Row shifting operation: the State is conceptually a 4x4 byte matrix filled column-by-column. ShiftRows cyclically shifts each row left by an offset equal to its row index: row 0 is unshifted, row 1 shifts left by 1 byte, row 2 by 2 bytes, and row 3 by 3 bytes. `test_shift_row.v` implements this purely by re-wiring bytes with assign statements (no arithmetic is needed — it is a fixed

permutation) and then registering the result. `inv_shift_row.v` performs the mirror-image right-shift for decryption.

How diffusion is achieved: `SubBytes` alone only changes bytes in place — it does not mix bytes across columns. `ShiftRows` breaks up the column structure by moving bytes from one column into another. Combined with `MixColumns`, this ensures that after just a couple of rounds, every output byte of the cipher depends on every input byte and every key byte — the diffusion property required for a secure block cipher.

3.7 MixColumns

Galois Field multiplication: `MixColumns` treats each column of the State as a 4-term polynomial over $GF(2^8)$ and multiplies it by a fixed polynomial modulo $x^4 + 1$. In hardware, this reduces to multiplying individual bytes by the constants 2 and 3 (forward) or 9, 11 (0x0B), 13 (0x0D), and 14 (0x0E) (inverse). Multiplication by 2 (xtime) is implemented in `multiply_2.v` as a left shift, with a conditional XOR by the irreducible polynomial 0x1B whenever the shifted-out bit was 1 — this is the standard $GF(2^8)$ modular reduction step using AES's reduction polynomial $x^8 + x^4 + x^3 + x + 1$. `multiply_3.v` derives multiply-by-3 as `multiply_2(x) XOR x`, since $3 = 2 \text{ XOR } 1$ in this field's arithmetic. The inverse multipliers (`mul_09`, `mul_0b`, `mul_0d`, `mul_0e`) are all built by cascading `multiply_2` three times to obtain x^2 , x^4 , and x^8 , then XOR-combining the needed powers (e.g., $14 = 8 \text{ XOR } 4 \text{ XOR } 2$, $9 = 8 \text{ XOR } 1$).

Matrix multiplication: each output byte of a column is a fixed XOR-combination of the four input bytes of that column, scaled by the appropriate constants. `test_column_mixing.v` computes all four output bytes for all four columns in parallel (16 outputs total) using 8 small multiplier instances per column, then XORs them together per the matrix definition, before registering the full new State.

Confusion/diffusion contribution: while `SubBytes` provides confusion and `ShiftRows` provides cross-column diffusion, `MixColumns` provides intra-column diffusion by ensuring a single-byte change in any row of a column affects all four bytes of that column's output. `MixColumns` is intentionally skipped in the final round (Section 5) because it adds no additional security once `SubBytes` and `ShiftRows` of the final round have already been applied, and removing it simplifies the corresponding decryption step.

3.8 AddRoundKey

XOR operation with round keys: this is the only step in AES where the secret key actually enters the computation. Each of the 16 State bytes is bitwise-XORed with the corresponding byte of the current round key. `add_round_key.v` slices both the 128-bit State and the 128-bit round key into sixteen byte pairs, computes 16 parallel XORs, and registers the combined result. Because XOR is its own inverse, `AddRoundKey` is used completely unchanged in both the encryption and decryption datapaths — only the round key supplied to it differs.

4. Implementation Details

4.1 Module Hierarchy

```

AES (top)
|-- ENCRYPTION_new
|   |-- round1_key (key schedule: 10 round keys)
|   |   |-- original_key -> array_of_byte
|   |   |-- `sbox x40 (4 bytes/round x 10 rounds, SubWord)
|   |-- add_round_key x11 (initial + 10 rounds)
|   |-- test_sub_byte x10 -> sbox x16 each
|   |-- test_shift_row x10
|   |-- `test_column_mixing x9 -> multiply_2 / multiply_3
|-- DECRYPTION
|   |-- round1_key (own copy of key schedule)
|   |-- add_round_key x11
|   |-- inv_shift_row x10
|   |-- inv_sub_byte x10 -> inv_sbox x16 each
|   |-- `inv_column_mix x9 -> mul_09/0b/0d/0e -> multiply_2

```

Note: coulumn_mixing.v (an early combinational variant that internally instantiates test_shift_row) and rcon.v (an empty stub module) exist in the project folder but are not instantiated anywhere in the final AES.v hierarchy — they appear to be earlier development iterations kept for reference. A clean-up pass to remove unused files would improve project hygiene for a portfolio repository.

4.2 Inputs and Outputs

Port (AES.v)	Direction	Width	Description
clk	input	1	System clock; every register in the design updates on its positive edge.
clr	input	1	Asynchronous, active-high clear; forces all pipeline registers to 0.
sel	input	1	Output select: 0 = present cipher_text, 1 = present decrypt_text.
in	input	128	128-bit plaintext block fed into the encryption core.
key_in	input	128	128-bit AES-128 cipher key, shared by encryption and decryption.
out	output reg	128	Registered 128-bit result: either ciphertext or recovered plaintext.

4.3 Registers, Combinational Logic, and Clocking Style

- **Combinational logic:** byte-slicing of 128-bit buses (assign statements), the S-Box / inverse S-Box lookup tables, RotWord re-wiring, and all Galois Field multiplier chains (multiply_2, multiply_3, mul_09/0b/0d/0e) are purely combinational, described with assign or always @(*).

- **Sequential logic:** every structural stage — AddRoundKey, SubBytes wrapper, ShiftRows wrapper, MixColumns wrapper, and the key-schedule registers — ends in an always @(posedge clk or posedge clr) block that registers its result. This means the design uses output-registered (retimed) combinational blocks rather than one giant combinational cloud for the whole cipher, which is good synthesis practice: it breaks long paths into smaller pieces and lets Vivado timing-close each stage independently.
- **Reset style:** clr is asynchronous and active-high (sensitivity list includes posedge clr), and every register is reset to a known value (128'h00 or 8'h00) when asserted — a standard, synthesis-friendly async-reset / sync-release style.
- **Latency consequence:** because every AES sub-step (not just every round) carries its own register, the forward cipher alone requires roughly 1 (initial AddRoundKey) + 9 x 4 (SubBytes, ShiftRows, MixColumns, AddRoundKey per round) + 3 (final round: SubBytes, ShiftRows, AddRoundKey) = 40 clock-edge stage transitions, plus the cipher_text output register and the top-level output mux register — i.e. roughly 41-42 clock cycles of pipeline latency from a fresh in/key_in to a valid out, in this non-looped, fully-unrolled architecture. This is an important design characteristic to call out in interviews/reports — see Section 8 for how this trades off against an iterative single-round datapath.

4.4 Notable Design Decisions

- **Fully unrolled, not iterative:** instead of looping a single round-function block 10 times with a round counter and a multiplexed key input, the design instantiates 10 physically distinct copies of the round hardware (n1..n41 in ENCRYPTION_new). This trades silicon area for simplicity of control logic — there is no FSM or round counter anywhere in the design.
- **Per-stage registers:** registering after every sub-operation (rather than only after every full round) keeps each combinational path short, which is friendly to FPGA timing closure, at the cost of extra latch cycles.
- **Symmetric key schedule duplication:** both ENCRYPTION_new and DECRYPTION instantiate their own independent copy of roun1_key. This avoids cross-module wiring complexity but duplicates the key-expansion hardware; sharing one key schedule between both cores (with the round-key order reversed for decryption) would reduce area.
- **Self-loopback verification:** as noted in Section 3.1, decryption is wired to always operate on the just-generated ciphertext. This is excellent for self-checking simulation testbenches but means the current top module cannot decrypt an arbitrary externally-supplied ciphertext without modification — a key candidate for the 'Future Improvements' list.

5. AES Encryption / Decryption Flow

5.1 Encryption (ENCRYPTION_new.v)

Stage	Operations Performed
Initial Round	AddRoundKey (State XOR original key)
Round 1 - 9	SubBytes -> ShiftRows -> MixColumns -> AddRoundKey (round key i)
Round 10 (Final)	SubBytes -> ShiftRows -> AddRoundKey (round key 10) -- MixColumns omitted

5.2 Decryption (DECRYPTION.v)

The decryption core implements the straightforward inverse cipher (round keys applied in reverse order, inverse operations applied in the mirrored sequence InvShiftRows -> InvSubBytes -> AddRoundKey -> InvMixColumns), which is mathematically valid because InvShiftRows and InvSubBytes commute (they each operate on independent bytes / independent positions):

Stage	Operations Performed
Initial Round	AddRoundKey with round key 10 (applied to the ciphertext)
Round 9 - 1	InvShiftRows -> InvSubBytes -> AddRoundKey (round key i) -> InvMixColumns
Final Round	InvShiftRows -> InvSubBytes -> AddRoundKey with the original key_in

Because this is the straightforward inverse cipher (rather than the FIPS-197 'Equivalent Inverse Cipher'), the round-key schedule does not need to be re-derived in a special order beyond simply being consumed from round 10 down to round 0 — exactly as wired in DECRYPTION.v (r10_key first, ..., key_in last).

6. Waveform Observations

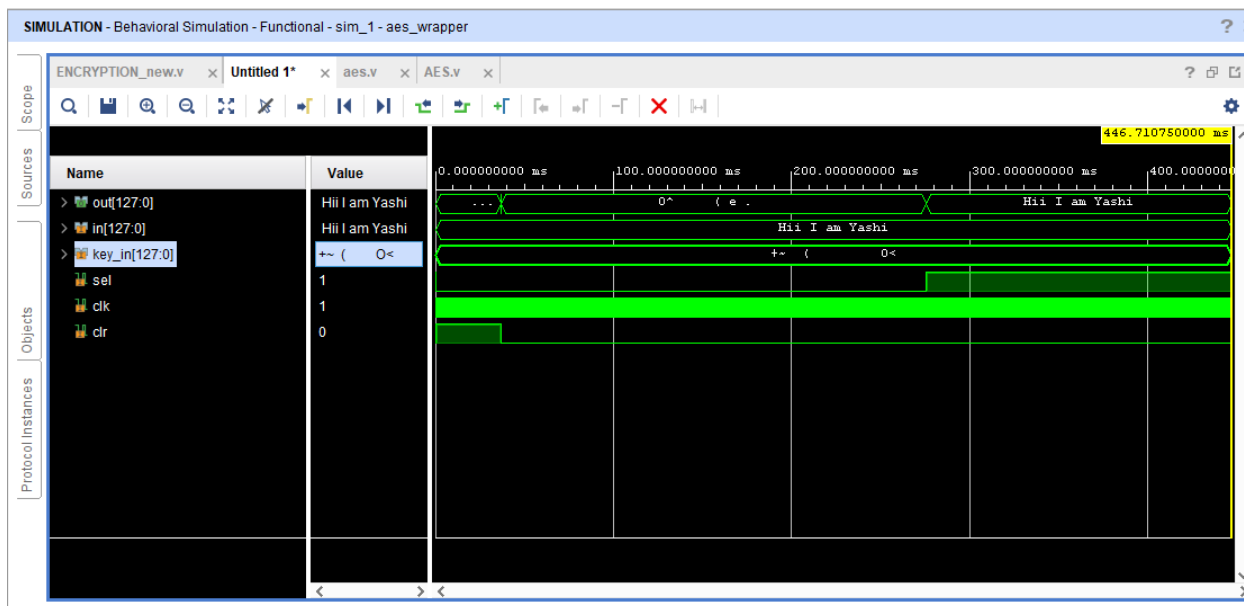
A Vivado simulation waveform was referenced for this report but has not yet been attached. Once you share the .wcfg screenshot or simulation log, I can fill in exact timing values, but the structural checklist below is what should be verified against your waveform, based on the architecture above:

Input: 486969204920616D20596173686920

After changing Radix to ASCII my input will be: Hii I am Yashi

Key_in: 2b7e151628aed2a6abf7158809cf4f3c

Key can be anything you want to set.



- **Reset behaviour:** on clr = 1, confirm out, cipher_text, decrypt_text, and all intermediate r*_add_key / r*_sub_byte / r*_row_shift / r*_cmix signals drop to 128'h00 asynchronously (i.e., the moment clr rises, not waiting for the next clk edge).
- **sel switching:** toggling sel should immediately (next clock edge) switch out between the live cipher_text and decrypt_text values without disturbing either internal pipeline.

7. Optimization and Future Improvements

- **Standalone decryption input:** add a dedicated ciphertext input port (and an input mux) so DECRYPTION can operate on externally supplied ciphertext instead of always being wired to ENCRYPTION_new's output — required for a general-purpose, reusable AES IP core.
- **Iterative (looped) round architecture:** replace the 10 unrolled, per-stage-registered round instances with a single round-function datapath reused 10 times via a round counter and a key-RAM/key-register-file. This trades 10x fewer clock cycles per encryption against a roughly 4x reduction in silicon area and routing — a classic area/throughput tradeoff well worth discussing in an interview.
- **True pipelining for higher throughput:** if the goal is maximum throughput rather than minimum area, the current per-stage registers are actually a good starting point — extend this into a fully pipelined architecture that can accept a new 128-bit block every clock cycle once the pipeline is full, by adding per-stage State buffering for multiple concurrent blocks (block-level pipelining) instead of relying on the unrolled rounds alone.
- **Shared key schedule:** compute the round-key schedule once (in the top module) and feed both ENCRYPTION_new and DECRYPTION from the same set of registered round keys, removing the current duplicate roun1_key instance and its associated S-Box/Rcon hardware.
- **Decryption module support files:** remove unused legacy modules (coulmn_mixing.v, the empty rcon.v stub, the duplicate 'original key.v'/original_key.v pair, test.v) from the synthesizable source set to keep the project clean for portfolio/GitHub presentation.
- **Support for AES-192 / AES-256:** parameterize the key length and round count (Nk/Nr per FIPS-197 Table 4) so the same datapath skeleton can target 12- or 14-round variants.
- **Implementing AES on board using Ethernet and WIFI Module:** move from simulation-only verification to an actual Vivado bitstream targeting a development board (e.g., Basys 3 / Nexys), using switches/UART for plaintext and key entry and LEDs/UART for ciphertext readout, then report post-implementation timing, resource utilization (LUTs, FFs, slices), and maximum achievable clock frequency (Fmax).
- **Side-channel protection:** for production-grade security silicon, evaluate masking the S-Box lookups and adding randomized clocking/dummy cycles to mitigate differential power analysis (DPA) and electromagnetic (EM) side-channel attacks, since this implementation's fixed S-Box LUT and unrolled datapath, while timing-deterministic, are not inherently power-balanced

8. References

- NIST FIPS PUB 197 — Advanced Encryption Standard (AES), November 2001
<https://nsarchive.gwu.edu/themes/custom/nsarchive/templates/pdfs/web/viewer.html?file=https%3A%2F%2Fnsarchive.gwu.edu%2Fsites%2Fdefault%2Ffiles%2Fdocuments%2F5989783%2FNational-Security-Archive-National-Institute-of.pdf>
- NIST SP 800-38A — Recommendation for Block Cipher Modes of Operation
- Xilinx Vivado Design Suite User Guide — Synthesis (UG901)
- FIPS 197 S-Box construction — $GF(2^8)$ multiplicative inverse + affine transform
- Daemen, J. & Rijmen, V. — The Design of Rijndael: AES – The Advanced Encryption Standard (Springer) .
- NESO Academy <https://www.youtube.com/watch?v=3MPkc-PFSRI>